

Object-Oriented Programming (OOP) in JavaScript

1. Introduction to OOP

What are OOPs and Why Do We Need Them?

Meaning: Object-Oriented Programming (OOP) is a programming paradigm centered around "objects" rather than just writing a long list of functions. It allows us to bundle data (properties) and actions (methods) together into a single structured unit.

Why we need it: As applications grow larger, keeping track of hundreds of variables and functions becomes chaotic. OOP helps us organize our code to be more readable, reusable, and easier to maintain. It mimics real-world entities, making complex logic simpler to understand.

2. Objects and Their Limitations

The Object Literal

In JavaScript, an object is a collection of key-value pairs.

```
Code:
const user1 = {
  name: 'Alice',
  score: 0,
  increment: function() {
    this.score++;
  }
};
```

The Problem with Manual Objects

Imagine you are creating a game and you need 1,000 users. If you manually create 1,000 object literals, you are repeating the same structure over and over again.

Memory Problem: If every object has an `increment` function inside it, JavaScript creates a brand new copy of that exact same function in memory 1,000 times! This wastes massive amounts of memory and slows down your application.

```
Code:
// Doing this 1,000 times manually is a nightmare and wastes
memory:
const user2 = { name: 'Bob', score: 0, increment: function() {
```

```
this.score++; } };  
const user3 = { name: 'Charlie', score: 0, increment:  
function() { this.score++; } };
```

3. Constructor Functions, `this`, and `new`

Before modern ES6 classes, developers used Constructor Functions to create multiple objects of the same type.

The `this` Keyword

Meaning: Inside a constructor or class, the `this` keyword refers to the specific instance of the object being created right now.

The `new` Keyword

Meaning: When you call a function with the `new` keyword, JavaScript does a few things behind the scenes:

1. It creates a brand new empty object: {}
2. It binds the `this` keyword to point to that new object.
3. It returns the new object automatically.

```
Code:  
function User(name) {  
  // 'this' refers to the new object being created  
  this.name = name;  
  this.score = 0;  
}  
  
// Using 'new' creates a fresh user object  
const userA = new User('Alice');  
const userB = new User('Bob');
```

4. ES6 Classes and The Memory Advantage

Meaning: In ES6 (2015), JavaScript introduced the `class` syntax. A class is basically a blueprint or a template for creating objects. It looks cleaner and more like traditional OOP languages (like Java or C++).

The `constructor` method

Meaning: The constructor is a special method inside a class that runs automatically exactly once when you create a new instance (using the `new` keyword). It is used to initialize the properties of the object.

The Memory Advantage of Classes

Unlike the manual objects we made earlier, when you define a method inside a Class, JavaScript does not copy that method for every single user.

Instead, it attaches the method to the Class's "Prototype". All 1,000 instances will share exactly ONE copy of the method in memory. This is highly efficient!

Code:

```
class Player {
  // The constructor runs automatically when 'new Player()'
  is called
  constructor(name) {
    this.name = name;
    this.score = 0;
  }

  // This method is stored ONCE in memory, shared by all
  players!
  incrementScore() {
    this.score++;
    console.log(`${this.name} scored! Total:
    ${this.score}`);
  }
}

const player1 = new Player('John');
const player2 = new Player('Sarah');

player1.incrementScore(); // John scored! Total: 1
player2.incrementScore(); // Sarah scored! Total: 1
```

5. The `static` Keyword

Meaning: Sometimes you want a method or property to belong to the Class itself, NOT to the individual objects created from it. We use the `static` keyword for this. Static methods are often used for utility functions.

Real-Life Example: Think of `Math.random()`. You don't need to create a `new Math()` object to use it; it belongs directly to the Math class.

Code:

```
class MathUtility {
  static add(a, b) {
    return a + b;
  }
}
```

```

}

// We call the method directly on the CLASS, not an instance.
console.log(MathUtility.add(5, 10)); // 15

// const mathObj = new MathUtility();
// mathObj.add(5, 10); // ERROR! mathObj.add is not a function

```

6. Inheritance

Meaning: Inheritance allows one class (Child) to inherit the properties and methods of another class (Parent). This promotes code reusability. We use the `extends` keyword.

```

Code:
class Animal {
  constructor(name) {
    this.name = name;
  }
  eat() {
    console.log(`${this.name} is eating.`);
  }
}

// Dog inherits from Animal
class Dog extends Animal {
  // Dog automatically gets the eat() method!
}

// Cat inherits from Animal
class Cat extends Animal {
}

const myDog = new Dog('Buddy');
myDog.eat(); // "Buddy is eating."

```

7. The `super` Keyword and Constructors in Inheritance

Meaning: If a child class has its own constructor, it **MUST** call `super()` before using the `this` keyword. `super()` calls the constructor of the Parent class, ensuring the parent properties are set up correctly.

```

Code:
class Animal {
  constructor(name) {
    this.name = name;
  }
}

```

```

    }
}

class Dog extends Animal {
  constructor(name, breed) {
    // Call the parent (Animal) constructor FIRST
    super(name);
    // Now we can use 'this' for Dog's specific properties
    this.breed = breed;
  }
}

const myDog = new Dog('Rex', 'German Shepherd');
console.log(myDog.name); // Rex
console.log(myDog.breed); // German Shepherd

```

8. Method Overriding

Meaning: If a Child class creates a method with the EXACT SAME NAME as a method in the Parent class, the Child's method will override (replace) the Parent's method when called on the child.

```

Code:
class Animal {
  speak() {
    console.log("This animal makes a sound.");
  }
}

class Dog extends Animal {
  // This OVERRIDES the Animal's speak method
  speak() {
    console.log("Woof! Woof!");
  }
}

const genericAnimal = new Animal();
genericAnimal.speak(); // "This animal makes a sound."

const myDog = new Dog();
myDog.speak(); // "Woof! Woof!"

```

9. Assignment: Practice Creating Models

Task: Create the following class-based models in JavaScript.

Model 1: Employee and Developer (Inheritance)

Task: Create a base class named Employee with a constructor that initializes name and salary. Include a method named work() that logs "[name] is working." to the console. Create a subclass named Developer that extends Employee. The subclass should have a constructor that accepts name, salary, and an additional property programmingLanguage.

Add a method named writeCode() inside the Developer class that logs "[name] is coding in [programmingLanguage]." to the console.

Instantiate the Developer class with the name, salary, and language. Finally, invoke both the work() and writeCode() methods on this instance.

Code:

```
class Employee {
    constructor(name, salary) {
        this.name = name;
        this.salary = salary;
    }
    work() {
        console.log(`${this.name} is working.`);
    }
}

class Developer extends Employee {
    constructor(name, salary, programmingLanguage) {
        super(name, salary);
        this.programmingLanguage = programmingLanguage;
    }
    writeCode() {
        console.log(`${this.name} is coding in
${this.programmingLanguage}.`);
    }
}

const dev = new Developer("Mohd. Umar Siddiqui", 85000,
"JavaScript");
dev.work();
dev.writeCode();
```

Model 2: BankAccount

Code:

```
class BankAccount {
    constructor(accountHolder, initialBalance) {
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }
}
```

```

    }
    deposit(amount) {
        this.balance += amount;
        console.log(`Deposited ${amount}. New balance:
${this.balance}`);
    }
    withdraw(amount) {
        if(amount <= this.balance) {
            this.balance -= amount;
            console.log(`Withdrew ${amount}. Remaining:
${this.balance}`);
        } else {
            console.log("Insufficient funds!");
        }
    }
}

```

Model 3: Vehicle and Car

Code:

```

class Vehicle {
    constructor(brand) {
        this.brand = brand;
    }
    startEngine() {
        console.log(`${this.brand} engine started.`);
    }
}

class Car extends Vehicle {
    constructor(brand, model) {
        super(brand);
        this.model = model;
    }
    honk() {
        console.log(`${this.brand} ${this.model} goes BEEP!`);
    }
}

```

Model 4: LibraryBook

Task: Create a class named LibraryBook with a constructor that initializes title and author. It should also automatically set a property named isAvailable to true.

Add a method named `borrowBook()`. If the book is available, set `isAvailable` to false and log You borrowed "[title]".. If it is already checked out, log Sorry, "[title]" is currently checked out.

Add a method named `returnBook()` that sets `isAvailable` to true and logs You returned "[title]". Create an instance of the class for the book. Test your system by sequentially calling the functions on your instance to simulate borrowing the book twice and then returning it.

```
Code:
class LibraryBook {
  constructor(title, author) {
    this.title = title;
    this.author = author;
    this.isAvailable = true;
  }
  borrowBook() {
    if (this.isAvailable) {
      this.isAvailable = false;
      console.log(`You borrowed "${this.title}".`);
    } else {
      console.log(`Sorry, "${this.title}" is currently
checked out.`);
    }
  }
  returnBook() {
    this.isAvailable = true;
    console.log(`You returned "${this.title}".`);
  }
}
```

Model 5: ShoppingCart

Task: Create a class named `ShoppingCart` with a constructor that initializes an empty array called `items`.

Add a method named `addItem(itemName, price)` that pushes an object containing the name and price into the `items` array. It should log Added [itemName] to cart. to the console.

Add a method named `calculateTotal()` to sum up the prices of all items in the cart. It should log Total Price: \$[total] and return the total value.

Create an instance of the `ShoppingCart` class. Add three items to your cart: a "**Laptop**" for \$1200, a "**Mouse**" for \$25, and "**Headphones**" for \$80.

Finally, invoke the `calculateTotal()` method to verify the final sum.

Code:

```
class ShoppingCart {
  constructor() {
    this.items = [];
  }
  addItem(itemName, price) {
    this.items.push({ itemName, price });
    console.log(`Added ${itemName} to cart.`);
  }
  calculateTotal() {
    let total = this.items.reduce((sum, item) => sum +
item.price, 0);
    console.log(`Total Price: $$${total}`);
    return total;
  }
}
```